

DEV_IMPL_GUIDE - EM-CFG-04 Approval Workflow Catalog

Use with DD_EM-CFG-04-Approval_Workflow_Catalog-v1.0.md.

This guide explains how shared approval flows execute at runtime.

The key implementation rule is simple:

- EM-CFG-04 owns approval policy, tasks, decisions, and callbacks.
- The owning module owns the business record and final mutation.

1. Runtime Components

Component	Responsibility
Approval API	Receives policy/admin requests and runtime approval requests.
Approval service	Evaluates policies, creates requests/tasks, records decisions.
Approval DB schema	Stores policies, conditions, steps, requests, tasks, decisions, callback logs.
Camunda	Runs multi-step/timer/revision approval BPMN where configured.
Approval workers	Execute approval service tasks and callback delivery.
RBAC client	Checks approver permission/scope via EM-CFG-03.
ICN client	Sends internal approval task notifications.
Owning module callback	Receives final approved/rejected/revision outcome.
Kafka outbox dispatcher	Publishes approval events.

2. Before Requests Can Work

Create these tables:

```
approval_policy
approval_policy_condition
approval_step
approval_request
approval_task
approval_decision
approval_callback_log
idempotency_key
outbox_event
```

Deploy at least one reference BPMN if multi-step approvals are used:

```
approval_generic_single_step_v1
approval_generic_threshold_v1
approval_generic_two_step_v1
approval_generic_revision_loop_v1
```

Seed minimum policies:

```
KYC_L1_APPROVAL
KYC_L2_APPROVAL
BIL_ADJUSTMENT_APPROVAL
WALLET_MANUAL_ADJUSTMENT_APPROVAL
TAX_INVOICE_CANCEL_APPROVAL
HOMEPASS_STATUS_TRANSITION_APPROVAL
FRANCHISE_RETIREMENT_APPROVAL
```

3. Flow 1 - Owning Module Evaluates Approval Policy

Example: billing adjustment request.

3.1 Owning Module Has Already Created Pending Request

BIL-02-ADJ receives a request and creates its own row:

```
adjustment_request
status = PENDING_APPROVAL_EVALUATION
```

The module validates invoice/customer/amount/reason before calling approvals.

3.2 Module Calls Evaluate

```
POST /api/approvals/evaluate
Authorization: Bearer <service-token>
Content-Type: application/json

{
  "operatorCode": "WIK",
  "moduleCode": "BIL-02-ADJ",
  "actionCode": "invoice.adjustment.approve_flow",
  "requesterUserId": "kc_user_billing_01",
  "subjectType": "INVOICE_ADJUSTMENT",
  "subjectId": "adj_01J_1001",
  "context": {
    "amount": 7000,
    "currency": "KES",
    "reasonCode": "SERVICE_CREDIT",
    "franchiseId": "fr_01J_NRB_002"
  }
}
```

3.3 Approval Service Executes

1. Authenticate service token.
2. Validate required fields.
3. Find active policy by:
 - operator;
 - module;
 - action;
 - effective date.
4. Evaluate active conditions in condition_order.
5. Decide approval mode:
 - AUTO;
 - SINGLE_STEP;
 - MULTI_STEP;
 - BPMN.
6. Load active steps for selected step_set_code.
7. Return decision to owning module.

Response:

```
{
  "approvalRequired": true,
  "policyId": "apol_bil_adj_wik_v1",
  "approvalMode": "MULTI_STEP",
  "stepSetCode": "HIGH_VALUE",
  "camundaProcessKey": "approval_generic_threshold_v1",
  "steps": [
    {
      "stepOrder": 1,
      "stepCode": "BILLING_LEAD_APPROVAL",
      "requiredPermissionCode": "billing.adjustment.approve"
    },
    {
      "stepOrder": 2,
      "stepCode": "FINANCE_APPROVAL",
      "requiredPermissionCode": "billing.adjustment.approve"
    }
  ]
}
```

If approvalRequired = false, the owning module applies the business request itself. EM-CFG-04 does not apply it.

4. Flow 2 - Owning Module Starts Approval Request

4.1 Module Calls Start

```
POST /api/approvals/requests
Authorization: Bearer <service-token>
Idempotency-Key: approval-adj-01J-1001
Content-Type: application/json

{
  "operatorCode": "WIK",
  "policyId": "apol_bil_adj_wik_v1",
  "moduleCode": "BIL-02-ADJ",
  "subjectType": "INVOICE_ADJUSTMENT",
  "subjectId": "adj_01J_1001",
  "requesterUserId": "kc_user_billing_01",
  "context": {
    "amount": 7000,
    "currency": "KES",
    "reasonCode": "SERVICE_CREDIT",
    "franchiseId": "fr_01J_NRB_002",
    "summary": "Credit customer for outage."
  }
}
```

4.2 Approval Service Executes

1. Check idempotency key.
2. Load policy and selected step set.
3. Validate policy is active.
4. Insert approval_request with status = PENDING.
5. If policy uses Camunda:
 - start process instance with business key approval_request_id;
 - store camunda_process_instance_id.
6. If policy is simple DB-driven:
 - create first approval_task directly.
7. Insert outbox event ApprovalRequestStarted.
8. Insert outbox event ApprovalTaskCreated for the first task.
9. Call ICN or rely on event consumer to notify candidates.
10. Store idempotency response.
11. Commit.

Response:

```
{
  "approvalRequestId": "apr_01J_ADJ_1001",
  "status": "PENDING",
  "currentStepOrder": 1,
  "tasksCreated": 1
}
```

4.3 Camunda Variables

For BPMN-backed approvals, pass:

```
{
  "approvalRequestId": "apr_01J_ADJ_1001",
  "operatorCode": "WIK",
  "moduleCode": "BIL-02-ADJ",
  "subjectType": "INVOICE_ADJUSTMENT",
  "subjectId": "adj_01J_1001",
  "policyId": "apol_bil_adj_wik_v1",
  "stepSetCode": "HIGH_VALUE",
  "requesterUserId": "kc_user_billing_01"
}
```

Do not put large business payloads into Camunda variables. Store context in `approval_request.context_json` and load by id.

5. Flow 3 - Backoffice Loads Approval Tasks

5.1 User Opens Task Inbox

```
GET /api/approvals/tasks?assigneeOrCandidate=kc_user_finance_02&status=OPEN
Authorization: Bearer <jwt>
```

5.2 Approval API Executes

1. Validate JWT.
2. Load open tasks.
3. For each task, call/check EM-CFG-03 effective access:
 - required permission;
 - target scope from approval context;
 - requester self-approval rule.
4. Return only tasks the user can act on.

Response:

```
{
  "items": [
    {
      "approvalTaskId": "apt_01J_ADJ_STEP1",
      "approvalRequestId": "apr_01J_ADJ_1001",
      "moduleCode": "BIL-02-ADJ",
      "subjectType": "INVOICE_ADJUSTMENT",
      "subjectId": "adj_01J_1001",
      "stepCode": "BILLING_LEAD_APPROVAL",
      "displayName": "Billing Lead Approval",
      "dueAt": "2026-05-31T12:30:00Z",
      "summary": "Credit customer for outage.",
      "allowedActions": ["APPROVE", "REJECT", "REQUEST_REVISION"]
    }
  ]
}
```

Backoffice should call the owning module for full subject detail if the approver opens the task.

6. Flow 4 - Approver Acts On Task

6.1 Approver Sends Action

```
POST /api/approvals/tasks/apt_01J_ADJ_STEP1/approve
Authorization: Bearer <jwt>
Idempotency-Key: approve-task-apt-01J-step1
Content-Type: application/json

{
  "comment": "Invoice and outage notes reviewed."
}
```

6.2 Approval Service Validates

1. Validate JWT and get actor_user_id.
2. Check idempotency key.
3. Lock task row:

```
SELECT *
FROM approval_task
WHERE approval_task_id = :taskId
FOR UPDATE;
```
4. Validate task is OPEN or claimed by actor.

5. Load approval request and step.

6. If `allow_requester = false`, reject if `actor_user_id = requester_user_id`.

7. Call RBAC check:

```
{
  "operatorCode": "WIK",
  "authUserId": "kc_user_finance_02",
  "permissionCode": "billing.adjustment.approve",
  "target": {
    "scopeType": "FRANCHISE",
    "scopeValue": "fr_01J_NRB_002"
  }
}
```

8. If RBAC denies, return 403.

9. Insert `approval_decision`.

10. Mark task COMPLETED.

11. If Camunda-backed, complete the active Camunda user task with variable:

```
{
  "approverAction": "APPROVE",
  "approvalTaskId": "apt_01J_ADJ_STEP1",
  "actorUserId": "kc_user_finance_02"
}
```

12. If DB-driven, move to next step or finalize.

13. Insert outbox event `ApprovalTaskCompleted`.

14. Commit.

6.3 Response

```
{
  "approvalTaskId": "apt_01J_ADJ_STEP1",
  "status": "COMPLETED",
  "decision": "APPROVE",
  "approvalRequestStatus": "PENDING",
  "nextStepCreated": true
}
```

7. Flow 5 - Final Outcome Delivery

When all required steps approve, or when any step rejects/requests revision, EM-CFG-04 finalizes the approval request.

7.1 Approval Service Finalizes

1. Lock approval request.

2. Set status:

- APPROVED;
- REJECTED;
- REVISION_REQUESTED;
- EXPIRED;
- CANCELLED.

3. Insert outbox event for final outcome.

4. Insert callback log row with `status = PENDING`.

5. Commit.

7.2 Callback Worker Calls Owning Module

```
POST /internal/billing/adjustments/adj_01J_1001/approval-outcome
Authorization: Bearer <service-token>
Content-Type: application/json
```

```

{
  "approvalRequestId": "apr_01J_ADJ_1001",
  "operatorCode": "WIK",
  "moduleCode": "BIL-02-ADJ",
  "subjectType": "INVOICE_ADJUSTMENT",
  "subjectId": "adj_01J_1001",
  "outcome": "APPROVED",
  "decisions": [
    {
      "stepCode": "BILLING_LEAD_APPROVAL",
      "decision": "APPROVE",
      "actorUserId": "kc_user_finance_02",
      "decisionAt": "2026-05-29T13:00:00Z"
    }
  ]
}

```

Owning module executes idempotently:

1. Lock subject row.
2. If outcome already applied for same approvalRequestId, return success.
3. If approved, apply business mutation.
4. If rejected, set subject rejected.
5. If revision requested, set subject awaiting revision.
6. Store approval outcome reference.
7. Return success.

7.3 Callback Retry

If callback fails:

1. Update `approval_callback_log.status = FAILED_RETRYABLE`.
2. Retry with exponential backoff.
3. Alert after threshold, for example 10 failed attempts or 30 minutes.
4. Backoffice shows approval outcome as final but callback delivery as failed.
5. Admin can retry callback from repair screen.

Do not roll back the approval decision because callback failed.

8. Flow 6 - Request Revision

When approver asks for revision:

1. Insert decision `REQUEST_REVISION`.
2. Mark current task completed.
3. Set approval request `REVISION_REQUESTED`.
4. Deliver callback to owning module.
5. Owning module sets subject status to `REVISION_REQUESTED` or equivalent.
6. Requester edits and resubmits through owning module.
7. Owning module can start a new approval request or reopen same subject with a new approval request id.

Rule: use a new approval request for a materially changed business payload. This preserves audit clarity.

9. Flow 7 - Task Timeout

If policy step has `timeout_hours`:

1. Camunda timer or scheduled job finds overdue open task.

2. Mark task EXPIRED.
3. Apply policy timeout behavior:
 - escalate to next role/team;
 - auto-reject;
 - keep pending and notify supervisor.
4. Insert decision/audit row where applicable.
5. Emit ApprovalRequestExpired or ApprovalTaskExpired.
6. Notify ICN.

For v1, keep timeout behavior simple: notify and mark overdue; do not auto-approve.

10. SQL Migration Sketch

Conceptual schema:

```
CREATE TABLE approval_policy (
  policy_id text PRIMARY KEY,
  operator_code text NOT NULL,
  policy_code text NOT NULL,
  module_code text NOT NULL,
  action_code text NOT NULL,
  display_name text NOT NULL,
  status text NOT NULL,
  approval_mode text NOT NULL,
  camunda_process_key text,
  callback_url text NOT NULL,
  effective_from timestampz NOT NULL,
  effective_to timestampz,
  created_at timestampz NOT NULL,
  updated_at timestampz NOT NULL,
  UNIQUE (operator_code, policy_code)
);

CREATE TABLE approval_request (
  approval_request_id text PRIMARY KEY,
  operator_code text NOT NULL,
  policy_id text NOT NULL REFERENCES approval_policy(policy_id),
  module_code text NOT NULL,
  subject_type text NOT NULL,
  subject_id text NOT NULL,
  requester_user_id text NOT NULL,
  status text NOT NULL,
  current_step_order int,
  camunda_process_instance_id text,
  context_json jsonb NOT NULL DEFAULT '{}'::jsonb,
  created_at timestampz NOT NULL,
  updated_at timestampz NOT NULL,
  UNIQUE (operator_code, module_code, subject_type, subject_id, policy_id)
);

CREATE TABLE approval_task (
  approval_task_id text PRIMARY KEY,
  approval_request_id text NOT NULL REFERENCES approval_request(approval_request_id),
  step_id text NOT NULL,
  status text NOT NULL,
  candidate_permission_code text NOT NULL,
  candidate_role_code text,
  candidate_team_id text,
  claimed_by_user_id text,
  due_at timestampz,
  created_at timestampz NOT NULL,
  updated_at timestampz NOT NULL
);

CREATE TABLE approval_decision (
  decision_id text PRIMARY KEY,
  approval_request_id text NOT NULL REFERENCES approval_request(approval_request_id),
  approval_task_id text NOT NULL REFERENCES approval_task(approval_task_id),
  decision text NOT NULL,
  actor_user_id text NOT NULL,
  comment text,
  decision_at timestampz NOT NULL,
```

```

        metadata_json jsonb NOT NULL DEFAULT '{}':::jsonb
    );

```

Recommended indexes:

```

CREATE INDEX idx_approval_request_subject
ON approval_request(operator_code, module_code, subject_type, subject_id);

CREATE INDEX idx_approval_task_status
ON approval_task(status, due_at);

CREATE INDEX idx_approval_task_candidate_permission
ON approval_task(candidate_permission_code, status);

```

11. Error Handling

Case	Response
No active policy	200 from evaluate with approvalRequired = false or configured default; do not 500.
Multiple active policies for same action	409 CONFIGURATION_ERROR; admin must fix.
Start request with inactive policy	409.
Same idempotency key with different body	409.
User lacks approver permission	403.
Requester tries self-approval when blocked	403.
Task already completed	409 with current task status.
Callback endpoint unavailable	callback log FAILED_RETRYABLE; retry later.
Owning module rejects callback as unknown subject	callback log FAILED_FINAL and alert.

12. Required Tests

Test	Expected result
Evaluate matching active policy	Returns expected mode and steps.
Evaluate no policy	Returns no approval required or default policy decision.
Start approval request	Creates request, task, outbox events.
Duplicate start request	Same response, no duplicate request.
User with required permission approves	Decision stored and task completed.
User without permission approves	403, no decision row.
Requester self-approval blocked	403.
Final approval callback success	Callback log delivered.
Callback failure retry	Failed retryable then delivered on retry.
Rejection finalizes request	Owning module receives rejected outcome.
Revision request finalizes current approval	Owning module receives revision outcome.
Task timeout marks overdue/expired	Task no longer silently waits forever.

13. Implementation Checklist

1. Add approval schema migrations.
2. Seed minimum policies and steps.
3. Implement policy CRUD and activation.
4. Implement condition evaluation.
5. Implement approval evaluate API.
6. Implement start approval request API.

7. Implement task list and task action APIs.
8. Integrate EM-CFG-03 RBAC checks.
9. Integrate ICN task notifications.
10. Deploy reference BPMN processes if multi-step approvals are enabled.
11. Implement callback delivery worker and retry.
12. Add outbox events.
13. Patch BIL, ILM/FUL, RLM, EM-01, OSR, SIP docs as they adopt the catalog.

14. What Not To Build In V1

- Do not make EM-CFG-04 a universal business-state engine.
- Do not store full invoice/customer/stock records in approval context.
- Do not implement complex financial risk scoring here.
- Do not auto-approve on timeout.
- Do not bypass EM-CFG-03 for approver checks.
- Do not let callback failure erase a completed approval decision.